Augmenting VLAs with Spatial Co-Training Robot Data

Carl Brander
ETH Zurich
cbrander@student.ethz.ch

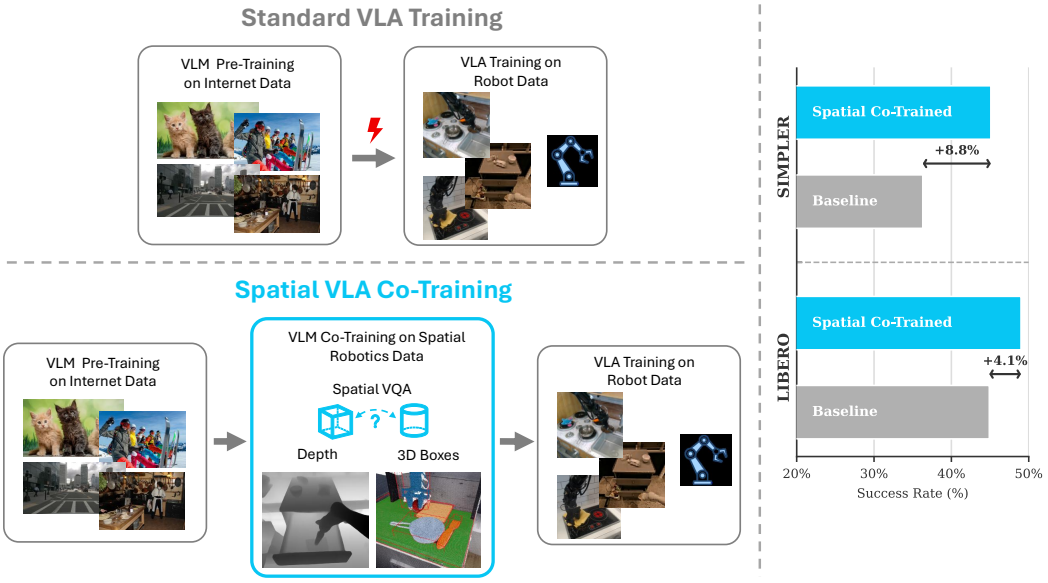


Figure 1: We introduce a low-cost, post data collection pseudo-labeling pipeline that augments monocular robot datasets with spatial supervision. Co-training VLM backbones on this spatially annotated data improves downstream VLA task success rates by up to 8.8% over a matched baseline.

Abstract: Vision-Language-Action (VLA) policies achieve strong generalization, but they are typically trained from robot datasets that lack explicit spatial supervision. We propose to co-train the vision-language backbone with pseudo-labeled spatial modalities (3D bounding boxes, depth, and spatial VQA) generated from monocular robot datasets using a modular, non-learned pipeline. This injects geometry-aware supervision before policy learning and is compatible with standard VLA training recipes. Across robotic manipulation benchmarks, spatial co-training consistently improves downstream success rates over a matched baseline without spatial supervision. Using all three spatial modalities, our co-trained models improve downstream success by 8.8 points on SIMPLER and 4.1 points on LIBERO. Ablations further show that depth supervision provides the most consistent single-modality gains across benchmarks. These findings suggest that inexpensive pseudo-spatial annotations can effectively scale VLA pretraining and complement recent co-training strategies in robot learning.

1 Introduction

Recent vision-language-action (VLA) models combine internet-scale visiolinguistic pretraining with imitation learning and show promising generalization for robot manipulation [1, 2, 3, 4]. This training recipe already yields policies that can follow diverse language instructions and transfer better than many task-specific controllers [1, 2, 3, 4]. Yet reliability remains limited in open-ended scenes: policies may understand task semantics but still fail to localize objects and execute spatially precise actions, for example by misjudging distances or relative object placement during grasping.

A key bottleneck is supervision. Internet-scale data provides rich semantic coverage but little robot-centric geometry, while robot datasets are costly, embodiment-specific, and typically lack explicit spatial annotations such as object extents, depth, and fine-grained relations [5, 6]. Consequently, many VLAs must infer 3D structure implicitly from sparse monocular robot datasets.

Recent co-training results suggest that combining heterogeneous modalities improves robustness and grounding [7, 2, 3], but two questions remain central for manipulation: which spatial signals are most useful, and how can they be obtained at scale without expensive sensors or manual 3D labeling.

In this work, we introduce a scalable co-training approach based on pseudo-spatial ground-truth for monocular robot datasets. We construct a modular, non-learned annotation pipeline that augments existing RGB trajectories with three complementary modalities: 3D bounding boxes, depth maps, and spatial visual question answering (VQA) pairs. It integrates them into VLM co-training before downstream VLA policy optimization. This design occupies a practical middle ground between generic semantic pretraining and costly, fully supervised 3D robotics datasets: it is cost-effective and broadly compatible with existing image datasets.

Overall, this paper makes three contributions: (i) a practical pseudo-labeling pipeline for spatial supervision at scale; (ii) a co-training recipe for injecting pseudo-spatial signals into VLM backbones; and (iii) an ablation-based analysis of modality-specific effects for downstream manipulation.

2 Related Work

Improving manipulation generalization through richer pretraining has become a central theme in recent robot foundation-model research [8]. SPEAR-1 introduces 3D-oriented pretraining to improve embodied generalization [2], Octo demonstrates the effectiveness of broad, diverse robot-data pretraining for generalist manipulation policies [1], and MolmoAct shows that depth-token supervision via a VQ-VAE pipeline can be integrated effectively into action models [7]. In parallel, eCoT and eCoT-lite show that distilling auxiliary modalities into a shared VLM representation improves policy quality [9, 10]. The large-scale study of Lin et al. [3] further demonstrates that co-training is especially beneficial under distribution shift. Our approach is closest in spirit to this line, but focuses specifically on pseudo-spatial modality co-training for manipulation.

A second line of work studies monocular 3D bounding box and pseudo-label generation. Cube R-CNN/Omni3D and 3D-MOOD provide strong open-domain monocular 3D detectors [11, 12], LabelAny3D improves large-scale 3D pseudo-annotation through an analysis-by-synthesis strategy [13]. Yet in our setting they transfer imperfectly to robotics scenes. Compared with these methods, our approach is a modular pipeline that composes existing foundation models (e.g., SAM 3, MoGe-2, and Qwen3-VL) for open-set object parsing and geometry estimation [14, 15, 16]. It is designed to transfer across both robotics and non-robotics datasets in a zero-shot setting while jointly producing three complementary spatial modalities. Finally, our VLA implementation follows the recent modular VLM4VLA framework for the flow-matching action head [17], and adopts Qwen2.5-VL-7B as the base VLM [18].

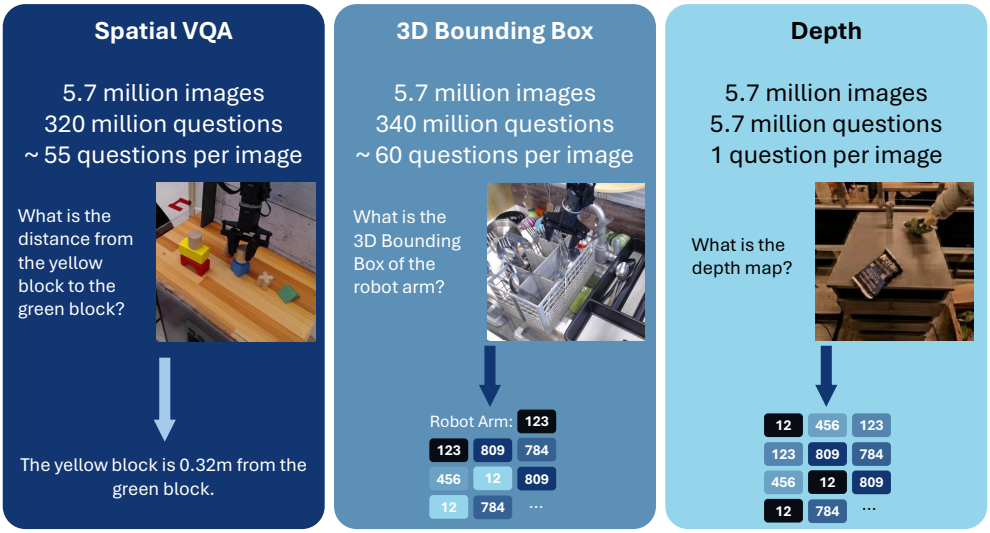


Figure 2: Our three spatial pseudo datasets generated from BridgeData V2 and Fractal (Open-X) robotics data [6, 5]. All spatial modalities are transformed into a VQA-style format for interleaved VLM co-training.

3 Supervision from Spatial Robotics Data

This section details how we construct pseudo-spatial supervision from monocular robotic datasets [6, 5, 19]. The objective is to produce scalable, post hoc, inter-scene-consistent targets for co-training without requiring depth sensors or human spatial and semantic annotations.

At a high level, given a monocular RGB frame of arbitrary resolution, our pipeline chains three foundation models: Qwen3-VL-4B for open-set scene parsing [16], SAM 3 for instance masks [14], and MoGe-2 for monocular geometry [15]. Their outputs are merged into a shared intermediate representation (objects, masks, depth, normals, and point-cloud features), from which we construct three training targets: 3D boxes, depth tokens, and spatial VQA.

We export each modality into separate shards so co-training can use any subset or all modalities jointly (Figure 2). For reproducibility, the exact question sets used to generate VQA-style spatial VQA, 3D bounding-box, and depth datasets from raw annotations are listed in Appendix Section 8.9.

3.1 3D Bounding Box Pseudo-Labels

Off-the-shelf monocular 3D detectors such as Cube R-CNN/Omni3D and 3D-MOOD are strong on their native benchmarks [11, 12], but we observe brittle behavior on robot-centric images. Figure 5 shows a representative qualitative comparison on a DROID scene [20]: baseline predictions frequently collapse to degenerate boxes with effectively zero overlap, while our pipeline preserves object extents and scene consistency. Appendix Figure 10 provides two additional qualitative comparisons on RH20T [21] and BridgeData V2 [6]. A single 3D bounding-box dataset example is shown in Appendix Figure 18.

After detecting and segmenting objects, we lift them into 3D using a pseudo-metric mono-depth estimate from MoGe-2 [15]. We then generate 3D box pseudo-labels with a surface-aligned fitting stage using RANSAC [22]. For each instance-specific point cloud, we estimate surfaces, enforce gravity-consistent orientation to the dominant scene surface (table for robotic manipulation scenes), and solve for a minimum-volume oriented box iteratively. Figure 3 summarizes the processing

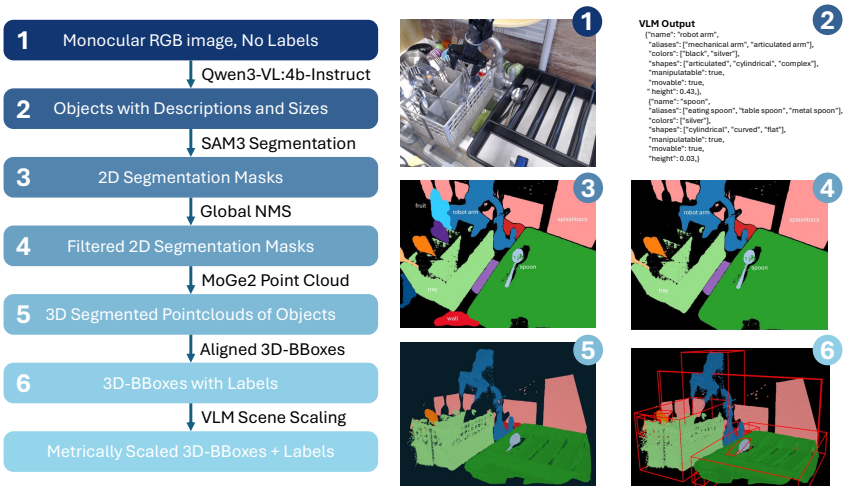


Figure 3: Detailed 3D bounding box pseudo-labeling pipeline. Starting from monocular RGB inputs, the pipeline combines VLM generated open-set object proposals, grounded segmentation masks, and monocular depth estimation to fit gravity-aligned minimum-volume 3D boxes. In addition to final box parameters, it exports auxiliary outputs including 2D masks and bounding boxes, depth maps, normal maps, object and scene point-clouds, camera intrinsics, and object-level textual attributes for downstream tasks.

order and intermediate outputs generated by this pipeline, and Appendix Figure 11 illustrates the alignment method.

To recover metric scale, we align measured object heights from the point cloud with VLM-provided size priors and estimate a robust median scale per dataset episode. Objects with scale-ratio outliers exceeding 50% relative to unit scale are filtered out, improving intra-scene consistency. Each final box uses 12 parameters, $[x, y, z, l, w, h, r_1, \ldots, r_6]$, with rotation encoded by a continuous 6D representation [23] that is more stable than Euler angles (Appendix Figure 12).

For location tokenization, x and y are clipped to $[-2.5, 2.5]$ m and z to $[0, 5]$ m. Each coordinate is discretized into 1024 bins ($\approx$5 mm per bin), and the same token set is reused across all 12 box parameters. The resulting tokenized 3D-box representation is converted into a VQA-style dataset, yielding $\approx$55 question-answer pairs per annotated image. The VLM vocabulary is extended with these 1024 location tokens, together with the delimiter tokens `<3dbbox_begin>` and `<3dbbox_end>`.

3.2 Depth Token Pseudo-Labels

Depth pseudo-annotations are derived from the shared 3D bounding box pipeline seen in Figure 3. The depth maps are scaled using the VLM-based scale parameter from Section 3.1. Next, a VQ-VAE depth tokenizer is trained on the pseudo-depth maps. The encoder maps each depth image to a compact latent representation, and vector quantization converts that representation into discrete code indices, following prior depth-token generation prior works [7]. A single depth dataset example is shown in Appendix Figure 19.

We use a codebook size of 1024 and compress each depth image into a sequence of 256 tokens. Accordingly, we add 1024 dedicated depth tokens to the VLM vocabulary, and the VLM is trained to represent each depth map with the same 256-token interface during co-training. The token-count trade-off that motivates this 256-token choice is shown in Appendix Figure 13. Exact VQ-VAE model-size/layout details and tokenizer training hyperparameters are provided in Appendix Tables 7 and 4. These indices become a token-level supervision modality that can be mixed with language in a VQA style dataset for the co-training stage. Qualitative examples of the resulting depth ground truth, VQ-VAE reconstruction, and corresponding VLM prediction are shown in Appendix Figure 14

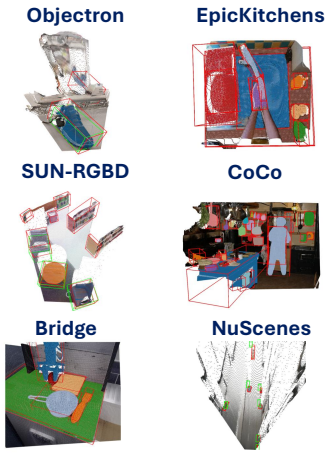


Figure 4: Pseudo-label quality on additional datasets, showing transfer beyond the core training distribution: Objectron, SUN RGB-D, and nuScenes samples from Omni3D [11]; EpicKitchens [24]; CoCo [25]; and BridgeData V2 [6]. Green 3D boxes indicate ground truth; red 3D boxes indicate predictions from our pseudo-labeling pipeline.

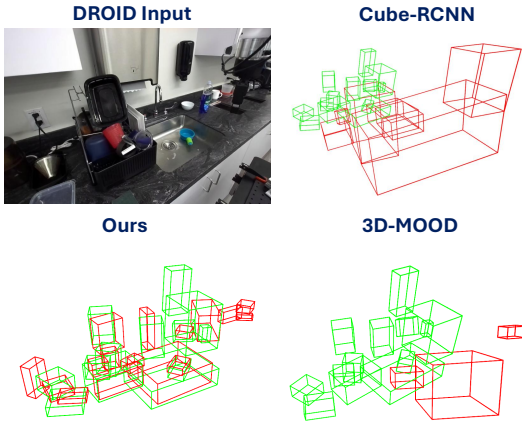


Figure 5: Comparison against alternative 3D bounding box detectors on DROID [20], showing improved robustness and spatial consistency of our 3D bounding box outputs.

3.3 Spatial VQA Pseudo-Labels

Spatial VQA targets are generated by permuting objects relations within each frame. We instantiate unary, pairwise, and listing typed question families that cover relative and absolute distances; left/right, front/behind, and above/below relations; size comparison; containment; and localization questions. Answers are computed directly from pseudo-geometry and auxiliary outputs of the 3D bounding box pipeline (boxes, depth, and attributes), then validated with consistency checks to remove ambiguous samples. The resulting dataset is stored in standard VQA format, enabling direct reuse with the same VLM tokenization and sampling infrastructure as other co-training data. A single spatial VQA dataset example is shown in Appendix Figure 20.

3.4 Pseudo-Label Quality and Failure Cases

Figure 5 provides representative qualitative results on robotic datasets, while Figure 4 shows transfer to diverse internet and driving-style imagery beyond a single scene distribution. Quantitative $\mathrm{AP}_{3D}(\mathrm{IoU@0.25})$ comparisons against 3D-MOOD and Cube R-CNN/Omni3D on both Omni3D and three separate robotic datasets are reported in Appendix Table 9.

Remaining failure modes include frontal views of objects where our pipeline cannot reliably infer object extent, as shown in Appendix Figure 17. As the VLM is the most influential part of this pipeline, any failure to list an object results in downstream failure, which makes VLM performance and selection crucial. We mitigate effects from VLM failures through confidence filtering, NMS tuning, and geometric sanity checks, but noisy pseudo-labels remain unavoidable.

4 Architecture and Training

Using the pseudo-spatial datasets introduced in Section 3, we adopt a two-stage training procedure: first co-training the VLM backbone, followed by optimization of the downstream VLA policy. This decomposition provides a practical balance between implementation complexity, computa-

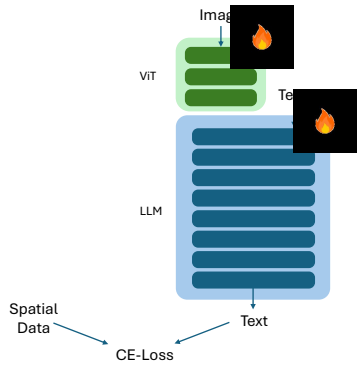


Figure 6: VLM backbone used for spatial co-training using a standard Cross-Entropy Loss in a teacher-forced training setting.

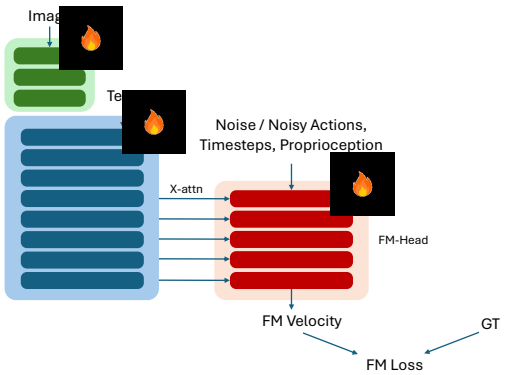


Figure 7: VLA policy architecture, where the VLM is connected to the flow-matching action head using cross attention on each transformer layer of the action head.

tional cost, and final task performance. Separating VLM co-training from VLA policy learning also facilitates benchmark-specific fine-tuning while generating intermediate spatially-enabled VLMs.

4.1 VLM Backbone Co-Training

We train four co-trained backbone variants that differ only in supervision modality: (i) 3D-box only, (ii) depth-token only, (iii) spatial-VQA only, and (iv) joint training with all three modalities. The VLM co-training architecture is illustrated in Figure 6. For location tokens of the 3D bounding box modality we utilize a soft gaussian CE-loss with a sigma schedule linearly reducing sigma from 5 to 0 through the first 10'000 training steps to provide a denser loss signal for near-miss token predictions as the discretized location token space is spatially coherent.

For the joint variant, we use a fixed sampling mixture of 55% 3D-box data, 35% depth-token data, and 10% spatial-VQA data. These weights are chosen empirically based on modality-specific learning speed and stability. Figure 2 summarizes the three co-training datasets, including dataset sizes and representative examples for each modality. Each variant is trained for 50'000 update steps, which aligned well with loss convergence. Exact run-level configurations are deferred to Appendix Table 2.

4.2 VLA Policy Learning

We transform the co-trained VLM into a VLA by attaching a modular flow-matching action head, following VLM4VLA [17]. At each of the last X action-head layers, action tokens cross-attend to the full VLM token sequence using the corresponding last X VLM hidden states. The resulting VLA architecture is shown in Figure 7.

The action head sequence uses action tokens, future-prediction tokens, and proprioceptive tokens. Temporal conditioning is injected through AdaLN at every layer. Exact layouts for the VLM backbone and VLA head are provided in Appendix Tables 5 and 6.

The resulting VLAs are trained on step-level RLDS robot trajectories from BridgeData V2 and Fractal (Open-X), or LIBERO mix depending on benchmark protocol [6, 5, 26]. We fine-tune a separate VLA per benchmark, pad Bridge proprioception at the second last position to match Fractal's eight-dimensional format, and train for 24'000 steps on SIMPLER or 15'000 on LIBERO, which matches loss convergence. Exact run statistics are reported in Appendix Table 3.

5 Experiments

5.1 Experimental Questions and Setup

We study three questions:
(Q1) Does spatial co-training with pseudo-labeled supervision improve downstream VLA manipulation performance over a matched baseline without spatial supervision?
(Q2) What is the contribution of each spatial modality (3D boxes, depth tokens, spatial VQA)?
(Q3) Are their effects complementary when combined?

We evaluate the fine-tuned multi-task VLAs on LIBERO and SIMPLER under their standard rollout protocols and task suites [26, 19]. Rollout counts per task are listed in Appendix Table 8. We report task success rate (%) by category (LIBERO: libero_10, libero_goal, libero_object, libero_spatial; SIMPLER: simpler_bridge, simpler_google), as summarized in Appendix Table 1. Checkpoints are selected through running a SIMPLER task subset of 50 rollouts on a list of the last saved checkpoints of the trained VLA (2k step checkpointing rate). We then take the three highest scoring checkpoints and run them through the full SIMPLER evaluation schedule. The best performing checkpoint is chosen to represent the VLA. The Pearson correlation coefficient between the 50-rollout SIMPLER subset score and full-benchmark SIMPLER success rate is $r = 0.682$ (Appendix Figure 15). At evaluation time, we use the empirically determined action chunk size 10, with execution of 4 actions before replanning. We include proprioception in both benchmarks, and apply dataset specific action-statistics normalization. Spatially co-trained variants are compared against a matched baseline with identical architecture, optimization, and inference settings.

5.2 Main Results

Figure 8 reports downstream success rates for the baseline VLA versus our VLAs with spatially co-trained VLM backbones. We see 8.8% and 4.1% success rate improvements in the SIMPLER and LIBERO benchmarks respectively for the VLAs using all three modalities during co-training. This answers Q1 in the affirmative: adding pseudo-spatial co-training improves downstream manipulation performance under matched architecture, optimization, and inference settings.

The gains are not uniform across task groups. On SIMPLER, improvements are concentrated in the *simpler_google* split, while *simpler_bridge* mostly decreases, except for the depth-only VLA; because SIMPLER contains far more Google tasks than Bridge tasks (Appendix Table 8), the aggregate benchmark effect remains strongly positive. On LIBERO, the largest improvements appear in *libero_goal* and *libero_10*. This could be explained by LIBERO Goal and LIBERO_10 being longer horizon and harder tasks where spatial scene understanding generates a larger benefit. Finally, the larger net gain on SIMPLER is consistent with stronger distributional overlap between our pseudo-labeled pretraining sources (Bridge and Fractal) and SIMPLER, and motivates the modality-level analysis below.

5.3 Modality Ablations

Single-modality ablations in Figure 8 answer Q2. Depth-only co-training is the most consistent single-modality variant, improving both SIMPLER and LIBERO totals over baseline. In contrast, 3D bounding box co-training underperforms on both benchmarks, suggesting weaker transfer under current pseudo-label quality and domain shift. Spatial-VQA-only co-training is benchmark-dependent: it improves LIBERO but is harmful on SIMPLER which invites for further investigation.

For Q3, the results indicate partial but meaningful complementarity. The all-three model is strongest on SIMPLER and remains competitive on LIBERO, yielding the best cross-benchmark balance overall. This suggests that combining modalities is beneficial in aggregate, even when one modality is less reliable in isolation.

Qualitatively, the co-trained all-modality VLM also predicts all three spatial outputs from RGB inputs (3D boxes, spatial VQA, and depth), as shown in Appendix Figure 16.

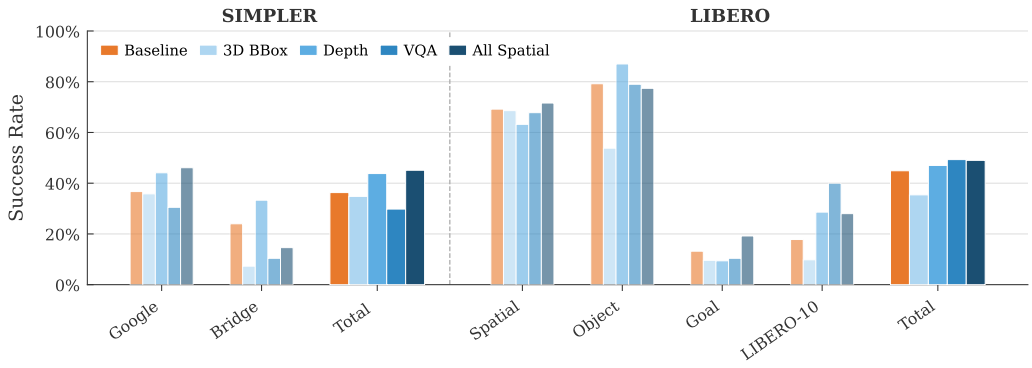


Figure 8: Visualization of the ablation results showing task success rates for both SIMPLER and LIBERO benchmarks corresponding to Appendix Table 1.

6 Conclusion

This work shows that pseudo-spatial co-training is a practical way to improve VLA manipulation without changing the base policy-learning recipe, as it can simply be employed between VLM pre-training and VLA fine-tuning. Joint training on depth, 3D boxes, and spatial VQA yields the strongest overall behavior, while modality ablations expose where transfer succeeds and where it breaks.

Beyond aggregate performance, our analysis identifies a central bottleneck: 3D-box supervision remains sensitive to real-to-sim shift (Appendix Figure 9). Improving pseudo-label robustness through more diverse spatially-annotated robot datasets and preserving spatial capabilities during policy fine-tuning are therefore key priorities for future VLA training.

7 Limitations and Future Work

Our analysis has three main limitations. First, pseudo-spatial supervision is noisier for 3D boxes and spatial VQA than for depth tokens, because both depend more directly on upstream object detection and label generation of the VLM in the pseudo-annotation pipeline. This likely weakens their standalone downstream gains. Second, the current two-stage recipe, while convenient, fully fine-tunes the VLM during VLA training without explicit capability-preservation mechanisms. This can attenuate benefits learned during co-training via forgetting [4, 3]. Third, our study does not include a targeted OOD stress-test protocol or real-world evaluations, even though recent co-training work indicates that the largest gains are from generalization, which appears under shift or in novel scenarios [3, 9, 10].

Future work should focus on four directions: *(i)* more robust pseudo-label generation and filtering through consistency checks and other pipeline improvements; *(ii)* labeling and co-training on more diverse robot data from both real and simulated domains to reduce real-to-sim transfer failures on simulator benchmarks; *(iii)* training strategies that mitigate forgetting, including co-training during VLA optimization, and eCoT-lite-style auxiliary supervision that connect the spatial skillset of the VLM with the robot control skills being learned; and *(iv)* closed-loop real-robot validation in novel scenes with targeted OOD stress tests.

Acknowledgments

I thank Marc Pollefeys, Oier Mees, Jeffrey Delmerico, and Isar Meijer for their exceptional guidance and supervision throughout this work. I also thank Liam Achenbach and Jonas Pai for their contributions to the shared training codebase and infrastructure as well as their guidance and help. This research was supported by computational resources provided by the Microsoft Spatial AI Lab and the CVG Lab at ETH Zurich

References

[1] Octo Model Team. Octo: An open-source generalist robot policy, 2024. URL https://arxiv.org/abs/2405.12213.

[2] N. Nikolov, G. Albanese, S. Dey, A. Yanev, L. V. Gool, J.-N. Zaech, and D. P. Paudel. Spear-1: Scaling beyond robot demonstrations via 3d understanding, 2025. URL https://arxiv.org/abs/2511.17411.

[3] F. Lin, K. Arora, J. Mercat, H. Nishimura, P. Shah, C. Xu, M. Zhang, M. Zolotas, M. Angeles, O. Pfannenstiehl, A. Beaulieu, and J. Barreiros. A systematic study of data modalities and strategies for co-training large behavior models for robot manipulation, 2026. URL https://arxiv.org/abs/2602.01067.

[4] Physical Intelligence. π0.5: a Vision-Language-Action model with open-world generalization, Apr. 2025. URL https://www.pi.website/download/pi05.pdf. Technical report.

[5] Open X-Embodiment Collaboration. Open x-embodiment: Robotic learning datasets and RT-X models, 2023. URL https://arxiv.org/abs/2310.08864.

[6] K. Pertsch, K. Gurevich, T. Yu, A. Mandlekar, M. J. Fu, C. Finn, and S. Levine. Bridgedata v2: A dataset for robot learning at scale, 2023. URL https://arxiv.org/abs/2308.12952.

[7] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. Molmoact: Action reasoning models that can reason in space, 2025. URL https://arxiv.org/abs/2508.07917.

[8] S. Grover, A. Gopalkrishnan, B. Ai, H. I. Christensen, H. Su, and X. Li. Enhancing generalization in vision-language-action models by preserving pretrained representations. *arXiv preprint arXiv:2509.11417*, 2025. URL https://arxiv.org/abs/2509.11417.

[9] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine. Robotic control via embodied chain-of-thought reasoning, 2024. URL https://arxiv.org/abs/2407.08693.

[10] W. Chen, S. Belkhale, S. Mirchandani, O. Mees, D. Driess, K. Pertsch, and S. Levine. Training strategies for efficient embodied reasoning, 2025. URL https://arxiv.org/abs/2505.08243.

[11] G. Brazil, A. Kumar, J. Straub, N. Ravi, J. Johnson, and G. Gkioxari. Omni3d: A large benchmark and model for 3d object detection in the wild. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13154–13164, 2023.

[12] Y.-H. Yang, L. Piccinelli, M. Segu, S. Li, R. Huang, Y. Fu, M. Pollefeys, H. Blum, and Z. Bauer. 3d-mood: Lifting 2d to 3d for monocular open-set object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. URL https://arxiv.org/abs/2507.23567.

[13] J. Yao, R. M. Redoy, S. Elbaum, M. B. Dwyer, and Z. Cheng. Labelany3d: Label any object 3d in the wild. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://openreview.net/forum?id=skQ6Ir6yxq.

[14] M. Li and et al. SAM 3: Segment anything in motion, 2025. URL https://arxiv.org/abs/2511.16719.

[15] S. Liu, C. Li, and et al. MoGe-2: Accurate monocular geometry estimation for open-domain images, 2025. URL https://arxiv.org/abs/2507.02546.

[16] Q. Team. Qwen3-vl technical report, 2025. URL https://arxiv.org/abs/2511.21631.

[17] J. Zhang, X. Chen, Q. Wang, M. Li, Y. Guo, Y. Hu, J. Zhang, S. Bai, J. Lin, and J. Chen. Vlm4vla: Revisiting vision-language-models in vision-language-action models, 2026. URL https://arxiv.org/abs/2601.03309.

[18] S. Bai and et al. Qwen2.5-vl technical report, 2025. URL https://arxiv.org/abs/2502.13923.

[19] P. R. Sanketi, C. Chi, Y. Tang, J. Bautista, S. Dasari, S. Savarese, L. Fei-Fei, J. Wu, and R. Martín-Martín. Simpler: A simulated real-world benchmark for robust and efficient robotic manipulation, 2024. URL https://arxiv.org/abs/2405.05941.

[20] A. Khazatsky, S. Nair, S. Dasari, S. Karamcheti, A. Garg, S. Levine, O. Mees, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset, 2024. URL https://arxiv.org/abs/2403.12945.

[21] H.-S. Fang et al. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot, 2024. URL https://arxiv.org/abs/2403.08635.

[22] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981. doi:10.1145/358669.358692.

[23] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5745–5753, 2019.

[24] D. Damen, H. Doughty, G. M. Farinella, S. Fidler, A. Furnari, E. Kazakos, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray. Scaling egocentric vision: The EPIC-KITCHENS dataset. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 720–755, 2018.

[25] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[26] B. Liu, Y. Qin, B. Büchner, S. Kannan, S. Huang, M. J. Kochenderfer, C. Finn, and M. Pavone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL https://arxiv.org/abs/2306.03310.

8 Appendix

8.1 3D Box Real-to-Sim Transfer

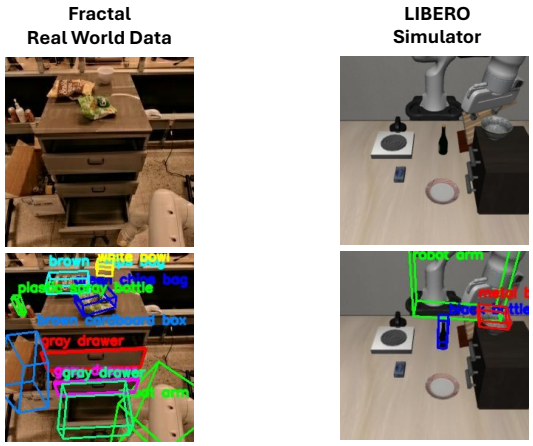


Figure 9: Real-to-sim transfer gap for 3D bounding-box predictions from the spatially co-trained VLM. Recall is substantially higher on real data than on LIBERO simulator images.

8.2 Additional RH20T and Bridge Qualitative Comparisons

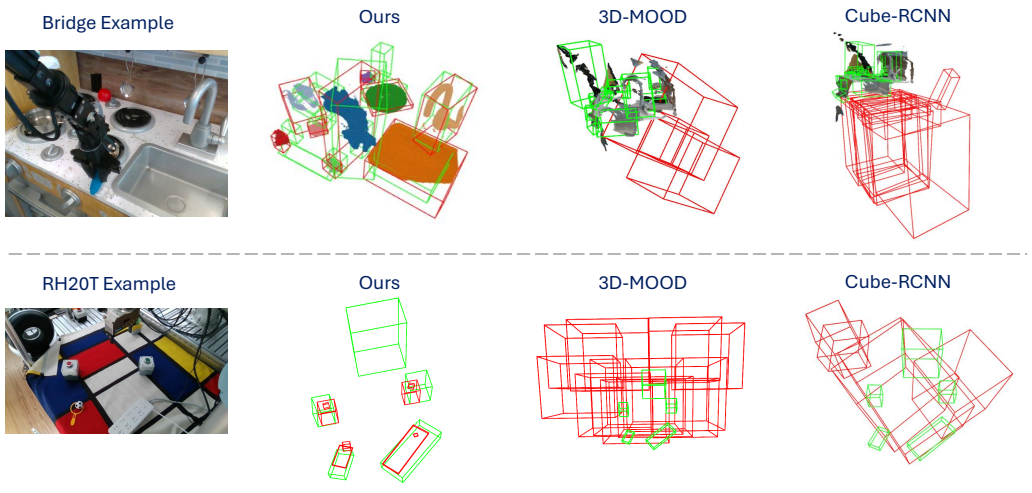


Figure 10: Additional qualitative comparisons against alternative 3D bounding-box detectors on RH20T [21] and BridgeData V2 [6]. As in the DROID example, baseline predictions often collapse, while our pipeline preserves object extents and scene consistency.

8.3 Main Benchmark Results Table

	SIMPLER			LIBERO				
Model setting	Google	Bridge	Total	Spatial	Object	Goal	libero_10	Total
Baseline	36.7%	24.0%	36.3%	69.2%	79.2%	13.2%	17.8%	44.9%
All 3 modalities	**46.1%**	14.6%	**45.1%**	**71.6%**	77.4%	**19.2%**	28.0%	49.0%
3D-Box only	35.8%	7.3%	34.8%	68.6%	53.8%	9.6%	9.8%	35.4%
Depth only	44.1%	**33.3%**	43.8%	63.2%	**87.0%**	9.4%	28.6%	47.0%
Spatial VQA only	30.5%	10.4%	29.8%	67.8%	79.0%	10.4%	**40.0%**	**49.3%**

Table 1: Main benchmark results across LIBERO and SIMPLER. All values are task success rates (%). Each row reports two separately fine-tuned VLAs that share the same backbone setting. All evaluations are run deterministically.

8.4 3D Box Alignment Method

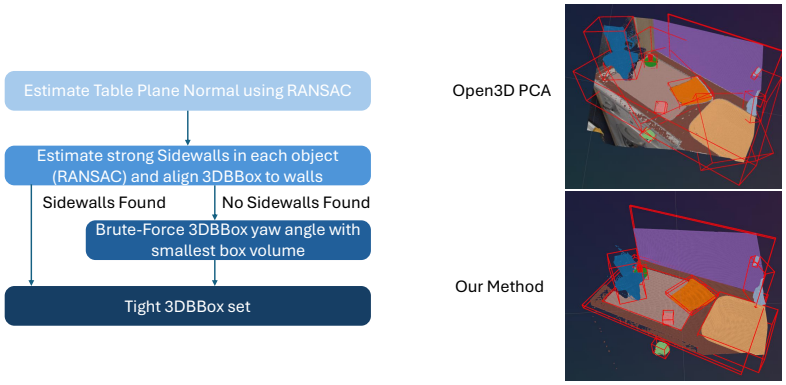


Figure 11: Illustration of the 3D bounding-box alignment method used during pseudo-label generation in comparison with generic Open3D PCA bounding box fitting.

8.5 6D Rotation vs. Euler Parameterization

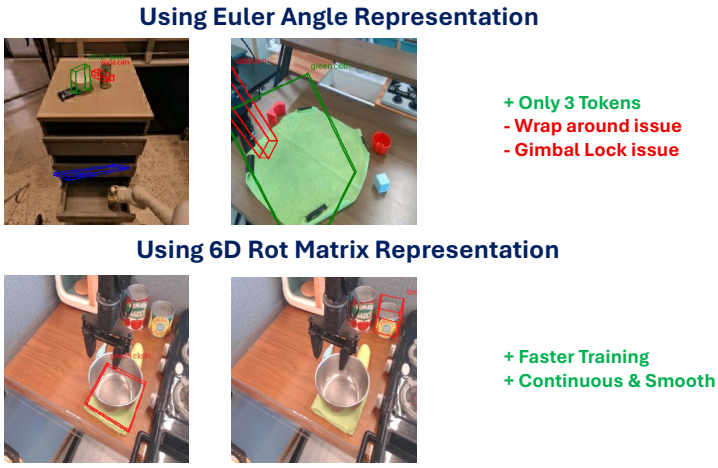


Figure 12: Comparison of 3D box rotation parameterizations. We use the continuous 6D representation because it yielded more stable optimization and better predictive behavior than Euler angles in our setting.

8.6 VQ-VAE Token Count Trade-off

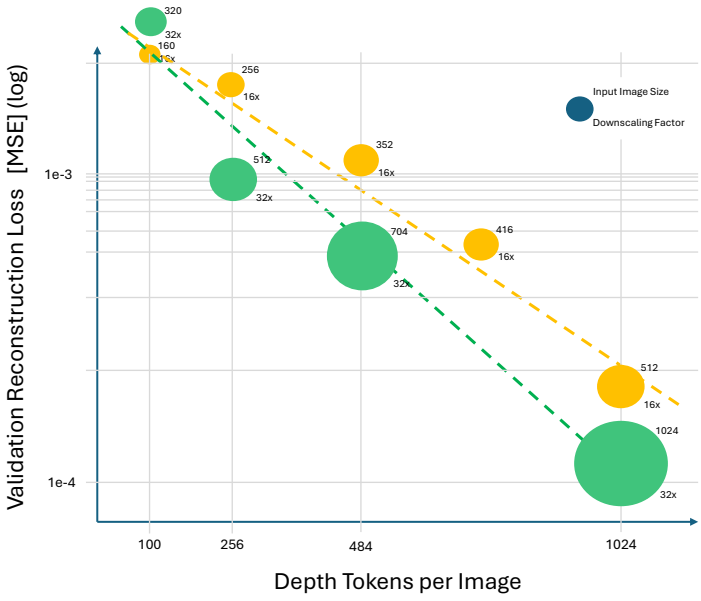


Figure 13: VQ-VAE token-count trade-off used to select 256 tokens per depth image. The more tokens chosen to represent an image the lower its reconstruction loss. But VLM training gets costlier. This is why a token count of 256 depth tokens per image with a vocabulary/codebook size of 1024 was chosen as a reasonable intermediate value.

8.7 Depth GT, VQ-VAE, and VLM Examples

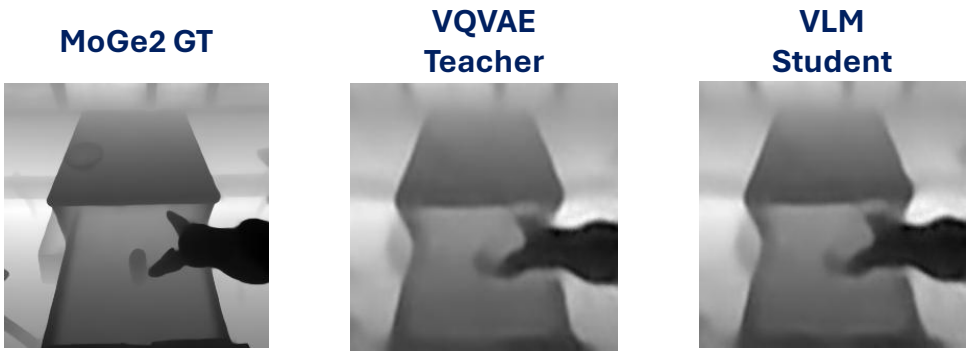


Figure 14: Qualitative depth examples showing pseudo depth ground truth from the MoGe-2 monodepth model, VQ-VAE reconstruction, and VLM depth prediction.

8.8 SIMPLER Subset Correlation for Checkpoint Selection

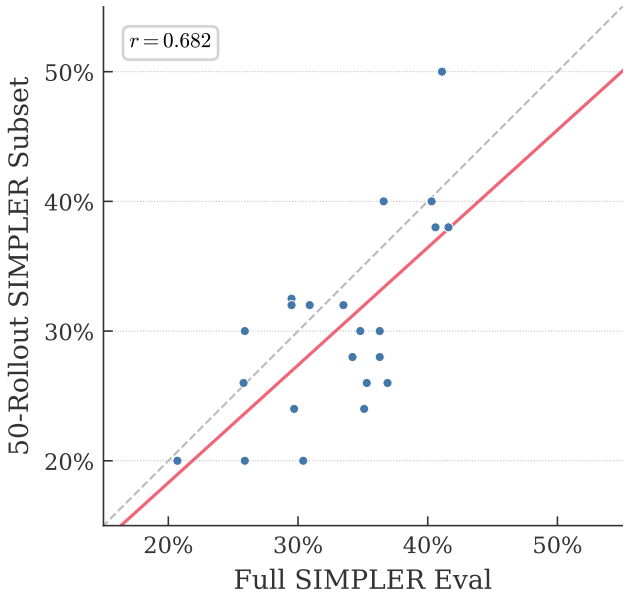


Figure 15: Correlation between checkpoint score on a 50-rollout SIMPLER subset and the full SIMPLER benchmark score, used for checkpoint selection.

8.9 Spatial Data to VQA-Type Co-Train Dataset Questions

Spatial VQA questions used for dataset generation.

- `object_distance`: What is the distance from the camera to the center of the {} in meters?
- `object_distance_between`: What is the distance from the center of the {} to the center of object {} in meters?
- `object_distance_sorted_closest_to_farthest`: Sort the following objects by their distance from the camera in meters. Start with the closest object: {}.

- `object_distance_sorted_farthest_to_closest`: Sort the following objects by their distance from the camera in meters. Start with the farthest object: {}.
- `object_size_binary_larger`: Is object {} larger than object {} in terms of volume?
- `object_size_binary_smaller`: Is object {} smaller than object {} in terms of volume?
- `spatial_relation_left`: Is object {} to the left of object {} from the camera's perspective?
- `spatial_relation_right`: Is object {} to the right of object {} from the camera's perspective?
- `spatial_relation_above`: Is object {} above object {} from the camera's perspective?
- `spatial_relation_below`: Is object {} below object {} from the camera's perspective?
- `spatial_relation_containment`: Is object {} contained within object {}?
- `which_is_closer_to_target`: Which object is closer to {}: {} or {}?
- `nearest_object_to_target`: What is the nearest object to {}?
- `objects_within_radius`: List all objects within {} meters of {}. Start with the closest.
- `second_closest_object_to_target`: What is the second closest object to {}?
- `are_objects_near_each_other`: Are {} and {} near each other (within {} meters)?
- `list_objects_left_to_right`: List all objects in the scene from left to right from the camera's perspective.
- `list_objects_top_to_bottom`: List all objects in the scene from top to bottom from the camera's perspective.
- `nth_object_from_direction`: What is the {} object from the {} side?
- `object_distance_binary_closer`: Is object {} closer than object {} to the camera in meters?
- `object_distance_binary_farther`: Is object {} further away than object {} to the camera in meters?
- `object_color`: What is the color of the object named {}?
- `object_aliases`: What are the aliases for the object named {}?
- `detect_objects_in_image`: List all names of objects visible in the image. List them alphabetically.
- `detect_color_objects_in_image`: List all names of objects visible in the image that are of color {}. List them alphabetically.
- `count_objects_by_name`: How many objects named {} are in the scene?
- `count_objects_by_color`: How many {} objects are in the scene?
- `has_multiple_of_name`: Are there multiple objects named {} in the scene?
- `which_names_have_duplicates`: Which object types appear more than once in the scene?

3D bounding-box questions used for dataset generation.

- `detect_3dbboxes`: List the 3D Bounding Boxes of all objects in the scene. Starting with the closest to the camera.
- `detect_3dbbox_from_name`: What ist the 3D Bounding Box of the {}?
- `detect_3dbbox_from_alias`: What ist the 3D Bounding Box of the {}?
- `detect_3dbbox_from_color`: What ist the 3D Bounding Box of the object with color {}?
- `detect_3dbboxes_from_names`: List the 3D Bounding Boxes of all objects named {}.

- `detect_3dbboxes_from_colors`: List the 3D Bounding Boxes of all objects with colors {}.
- `detect_3dbboxes_from_movable`: List the 3D Bounding Boxes of all movable objects.
- `detect_3dbboxes_from_manipulatable`: List the 3D Bounding Boxes of all manipulatable objects.
- `detect_3dbboxes_k_closest`: List the 3D Bounding Boxes of the {} closest objects to the camera, sorted from closest to farthest.
- `detect_3dbboxes_k_farthest`: List the 3D Bounding Boxes of the {} farthest objects to the camera, sorted from farthest to closest.
- `detect_3dbboxes_k_largest_volume`: List the 3D Bounding Boxes of the {} largest objects by volume, sorted from largest to smallest.
- `detect_3dbboxes_k_smallest_volume`: List the 3D Bounding Boxes of the {} smallest objects by volume, sorted from smallest to largest.
- `detect_3dbbox_closest_of_name`: What is the 3D Bounding Box of the closest {} to the camera?
- `detect_3dbbox_farthest_of_name`: What is the 3D Bounding Box of the farthest {} from the camera?
- `detect_3dbbox_largest_of_name`: What is the 3D Bounding Box of the largest {}?
- `detect_3dbbox_smallest_of_name`: What is the 3D Bounding Box of the smallest {}?
- `detect_3dbbox_leftmost_of_name`: What is the 3D Bounding Box of the leftmost {} from the camera's perspective?
- `detect_3dbbox_rightmost_of_name`: What is the 3D Bounding Box of the rightmost {} from the camera's perspective?

Depth questions used for dataset generation.

- `depth_map`: What is the depth map of the scene in raster order?

8.10 VLM Co-Training Run Configurations

Hyperparameter	3D-box only	Depth only	Depth + 3D-box	Spatial VQA only	All modalities
3D-box (%)	100	0	50	0	55
Depth (%)	0	100	50	0	35
Spatial VQA (%)	0	0	0	100	10
Steps	50'000	50'000	50'000	50'000	50'000
Warmup steps	2000	2000	2000	2000	2000
Learning rate	1×10^{-5}	1×10^{-5}	1×10^{-5}	1×10^{-5}	1×10^{-5}
LR scheduler	cosine (50000 steps)	cosine (50000 steps)	cosine (50000 steps)	cosine (50000 steps)	cosine (50000 steps)
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW
Weight decay	1×10^{-2}	1×10^{-2}	1×10^{-2}	1×10^{-2}	1×10^{-2}
Loss (3D-box location tokens)	CE + soft Gaussian CE	CE + soft Gaussian CE	CE + soft Gaussian CE	CE + soft Gaussian CE	CE + soft Gaussian CE
Optimizer	AdamW	AdamW	AdamW	AdamW	AdamW
Batch size	256	256	256	256	256
GPUs	64xGH200	64xGH200	64xGH200	64xGH200	64xGH200
Training Time	24h	24h	24h	24h	24h

Table 2: VLM co-training hyperparameters and training recipe.

8.11 VLA Training Run Configurations

Hyperparameter	LIBERO finetuned VLA	SIMPLER finetuned VLA
Robot training data	LIBERO mix	BridgeData V2 + Fractal (from Open-X)
Steps	14'000	24'000
Learning rate FM-head	2×10^{-5}	2×10^{-5}
Learning rate VLM Backbone	1×10^{-5}	5×1^{-5}
Batch size	512	512
GPUs	32xGH200	32xGH200
Training Time	7h	12h

Table 3: VLA training configurations per evaluation benchmark. We use the same recipe for LIBERO and SIMPLER VLA training, except for the number of steps to convergence due to differences in dataset size and variance.

8.12 VQ-VAE Depth Tokenizer Run Configurations

Hyperparameter	Depth tokenizer (VQ-VAE)
Training data	Our Pseudo depth map dataset
Learning rate	1e-4
Warmup steps	1000
LR scheduler	Exp
Optimizer	AdamW
Weight decay	1e-2
Batch size	256
GPUs	1xRTX5090
Reconstruction loss	MSE

Table 4: Depth tokenizer (VQ-VAE) training hyperparameters used for depth-token supervision.

8.13 VLM Model Layout Specifications

Component	Value
ViT	
HF checkpoint	Qwen/Qwen2.5-VL-7B-Instruct
Hidden size	1280
Number of transformer layers	32
Attention heads	16
Intermediate size	3'420
Output size	3'584
Patch size	14
LLM	
HF checkpoint	Qwen/Qwen2.5-VL-7B-Instruct
Hidden size	3'584
Number of transformer layers	28
Attention heads	28
Intermediate size	18'944
Base used vocab size	151'655
Extended vocab size	153'717

Table 5: VLM backbone size exactly follows Qwen2.5-VL-7B-Instruct implementation on Huggingface, Qwen2.5-VL vocab size is 152'064 out of which 399 tokens are padding/not used. We add 2052 additional tokens to the vocabulary.

8.14 VLA Flow Matching Head Model Layout Specifications

Component	Value
Input feature size	3'584
Cross attention size	3'584
Hidden size	1'024
Number of transformer layers	12
Attention heads	16
Action dimension	7
Action horizon	10
Future token count	32
Proprio token count	8
Timestep embedding	AdaLN
Positional embedding	Sinusoidal
Number of inference timesteps in DiT	20

Table 6: VLA flow matching implementation according to VLM4VLA public codebase.

8.15 VQ-VAE Depth Tokenizer Model Layout Specifications

Component	Value
Input image size	256
Number of resnet blocks	4
Downsample ratio	16
Codebook size	1024
Tokens per depth image	256
Codebook dimension	512
Hidden dimension	32
Using normalization	True
Max depth value	5
Weight commitment loss	0.5
Latent grid shape	16x16
Loss type	MSE

Table 7: VQ-VAE architecture adapted from MolmoAct with increased number of tokens per image and increase model size for higher reconstruction capacity.

8.16 Evaluation Rollout Protocol

Benchmark	Task split	# Tasks	Rollouts per task
LIBERO	libero_10	10	50
LIBERO	libero_goal	10	50
LIBERO	libero_object	10	50
LIBERO	libero_spatial	10	50
SIMPLER	simpler_bridge	4	24
SIMPLER	simpler_google	336	*varies (1–60)*

Table 8: Evaluation rollout protocol for benchmark reporting. Standard LIBERO and SIMPLER setup with deterministic seed for repeatable results.

8.17 3D Box AP Comparison Across Datasets

Dataset	Ours	3D-MOOD	Cube-RCNN
KITTI	0.027	0.314	**0.326**
Objectron	0.158	**0.679**	0.508
SunRGBD	0.120	**0.238**	0.153
Hypersim	0.004	**0.091**	0.075
ARKit	0.084	**0.539**	0.417
nuScenes	0.016	**0.358**	0.301
Omni3D average	0.068	**0.370**	0.297
Bridge	**0.21**	0.00	0.00
DROID	**0.12**	0.00	0.00
RH20T	**0.14**	0.00	0.00
Robot Datasets average	**0.16**	0.00	0.00

Table 9: $\mathrm{AP}_{3D}(\mathrm{IoU@0.25})$ comparison of our pseudo-labeling pipeline against 3D-MOOD and Cube-RCNN across Omni3D-style datasets and robotic datasets. 3D-MOOD and Cube-RCNN perform exceptionally well on the test-set of the Omni3D Datasets they have been trained on, but underperform on any out-of-distribution datasets such as the small hand-labeled robot dataset subsets.

8.18 Co-Trained VLM Predicting Spatial Modalities

Figure 16: Qualitative examples of spatial outputs predicted by the co-trained all modality VLM from RGB input: 3D bounding-box prediction (left), spatial VQA prediction (middle) (Q: How far away is the corn from the sink?, A: The corn is 0.21m from the sink), and depth prediction (reconstructed using the VQ-VAE decoder) (right).

8.19 Failure Case Example

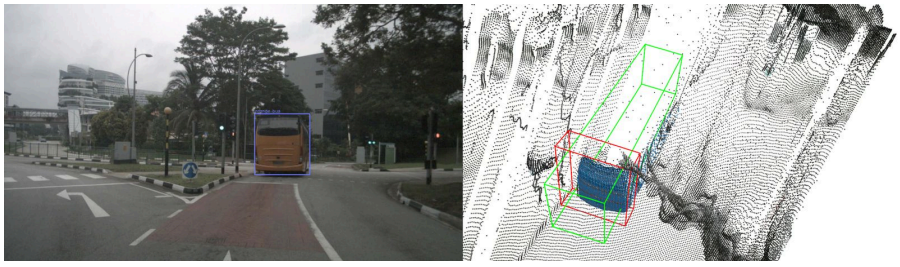
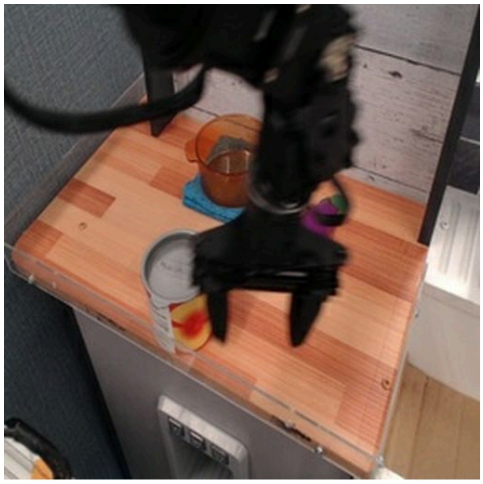


Figure 17: Example failure mode of our pseudo-labeling pipeline in a frontal-view configuration, where object extent estimation becomes ambiguous.

8.20 Single Data Examples for Each Pseudo Dataset



3D Bounding Boxes

Question:
List the 3D Bounding Boxes of all objects in the scene. Starting with the closest to the camera.

Answer:
The 3D bounding boxes are: <3dbbox_begin> "robot arm" <loc_509><loc_510><loc_80><loc_9><loc_42><loc_66><loc_545><loc_890><loc_169><loc_1><loc_540><loc_494> <3dbbox_end> <3dbbox_begin> "container" <loc_493><loc_521><loc_124><loc_16><loc_21><loc_16><loc_0><loc_508><loc_529><loc_525><loc_132><loc_854> <3dbbox_end> <3dbbox_begin> "work surface" <loc_506><loc_509><loc_145><loc_116><loc_71><loc_0><loc_969><loc_677><loc_352><loc_282><loc_852><loc_206> <3dbbox_end> <3dbbox_begin> "container" <loc_504><loc_494><loc_148><loc_35><loc_22><loc_10><loc_517><loc_134><loc_856><loc_1023><loc_507><loc_499> <3dbbox_end> <3dbbox_begin> "cloth" <loc_503><loc_502><loc_148><loc_24><loc_7><loc_1><loc_960><loc_689><loc_340><loc_265><loc_844><loc_211> <3dbbox_end>

Figure 18: Single data example from the 3D bounding-box pseudo dataset.

Depth Estimation

Question:
What is the depth map of the scene in raster order?

Answer:
The depth map tokens are: <depth_begin><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_900><depth_143><depth_143><depth_856><depth_143><depth_560><depth_411><depth_411><depth_411><depth_560><depth_560><depth_560><depth_560><depth_411><depth_560><depth_411><depth_856><depth_143><depth_900><depth_900><depth_900><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_469><depth_900><depth_143><depth_900><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_411><depth_900><depth_856><depth_143><depth_143><depth_411><depth_411><depth_411><depth_469><depth_143><depth_411><depth_411><depth_411><depth_560><depth_411><depth_411><depth_560><depth_469><depth_143><depth_143><depth_143><depth_560><depth_411><depth_560><depth_249><depth_96><depth_96><depth_249><depth_249><depth_96><depth_900><depth_411><depth_560><depth_856><depth_143><depth_143><depth_143><depth_560><depth_411><depth_856> ... <depth_end>

Figure 19: Single data example from the depth pseudo dataset.

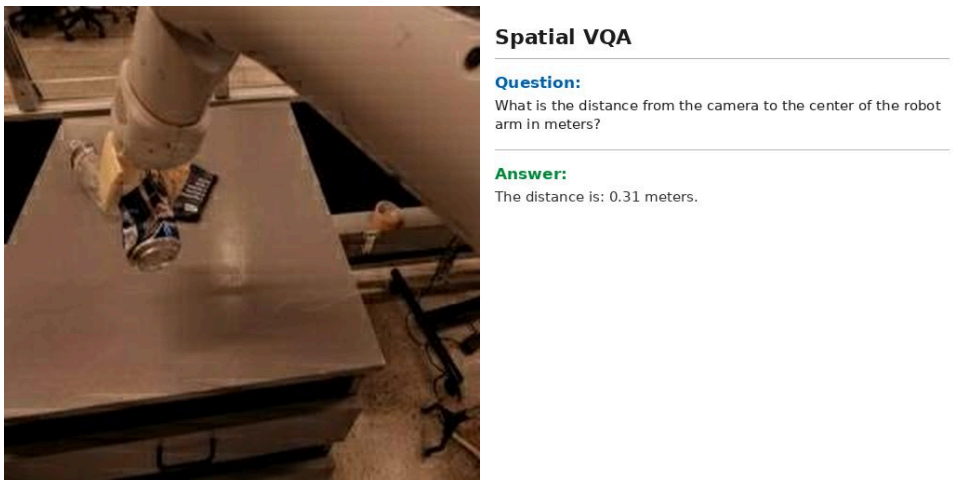


Figure 20: Single data example from the spatial VQA pseudo dataset.

8.21 Declaration of the Use of AI-Based Tools

AI-Based Tool	Use Case	Scope; remarks
GitHub Copilot	Coding	Used for coding assistance.
Grammarly	Corrections to the report	Used for grammar and style corrections in the manuscript.
ChatGPT Prism	Structuring and rephrasing	Used to structure sections in LaTeX rephrasing selected report text.
ChatGPT 5–5.4	Ideation and questions	Used for brainstorming and answering targeted writing questions.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Eigenständigkeitserklärung

Die unterzeichnete Eigenständigkeitserklärung ist Bestandteil jeder während des Studiums verfassten schriftlichen Arbeit. Eine der folgenden zwei Optionen ist **in Absprache mit der verantwortlichen Betreuungsperson** verbindlich auszuwählen:

☐ Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe, namentlich, dass mir niemand beim Verfassen der Arbeit geholfen hat. Davon ausgenommen sind sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson. Es wurden keine Technologien der generativen künstlichen Intelligenz[1] verwendet.

X Ich erkläre hiermit, dass ich die vorliegende Arbeit eigenverantwortlich verfasst habe. Dabei habe ich nur die erlaubten Hilfsmittel verwendet, darunter sprachliche und inhaltliche Korrekturvorschläge der Betreuungsperson sowie Technologien der generativen künstlichen Intelligenz. Deren Einsatz und Kennzeichnung ist mit der Betreuungsperson abgesprochen.

Titel der Arbeit:

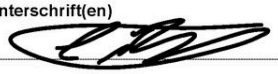
Augmenting VLAs with Spatial Co-Training Robot Data

Verfasst von:
Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich.

Name(n):	**Vorname(n):**
Brander	Carl

Ich bestätige mit meiner Unterschrift:
- Ich habe mich an die Regeln des «Zitierleitfadens» gehalten.
- Ich habe alle Methoden, Daten und Arbeitsabläufe wahrheitsgetreu und vollständig dokumentiert.
- Ich habe alle Personen erwähnt, welche die Arbeit wesentlich unterstützt haben.

Ich nehme zur Kenntnis, dass die Arbeit mit elektronischen Hilfsmitteln auf Eigenständigkeit überprüft werden kann.

Ort, Datum	**Unterschrift(en)**
Zurich, 13.03.26	

Bei Gruppenarbeiten sind die Namen aller Verfasserinnen und Verfasser erforderlich. Durch die Unterschriften bürgen sie grundsätzlich gemeinsam für den gesamten Inhalt dieser schriftlichen Arbeit.

[1] Für weitere Informationen konsultieren Sie bitte die Webseiten der ETH Zürich, bspw. https://ethz.ch/de/die-eth-zuerich/lehre/ai-in-education.html und https://library.ethz.ch/forschen-und-publizieren/Wissenschaftliches-Schreiben-an-der-ETH-Zuerich.html (Änderungen vorbehalten).